

PYCON US 2026

Breaking the Speed Limit

Statistician's workflow. Fast models.

Wenxin Jiang, Jian Yin | Biostatistics, CityU HK

Target first. Speed later.

Preserve statistical results.

Simulate for control.

Define baseline behavior.

Python orchestrates execution.

Match tools to bottlenecks.

WHY THIS TALK

Speed follows validation

Specify. Execute. Verify.

Target First

Define core statistic.

Executable Reference

Build matchable baseline.

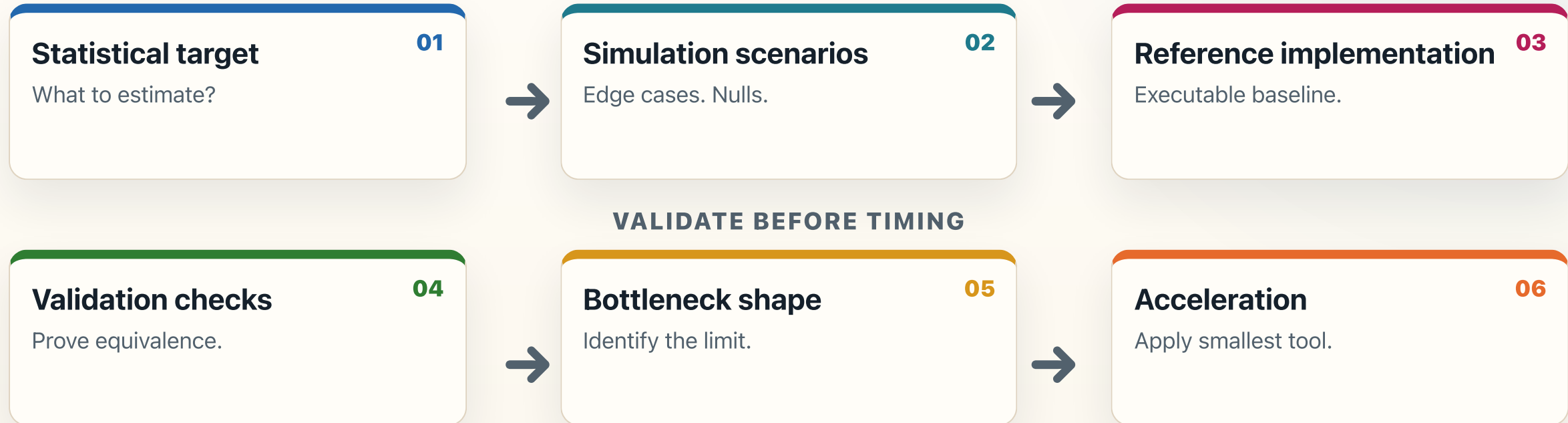
Conditional Speed

Answer identical question.

Speed only counts when the math is preserved.

Simulation creates an answer key

Control messy data behavior.



Simulation turns statistical behavior into tests

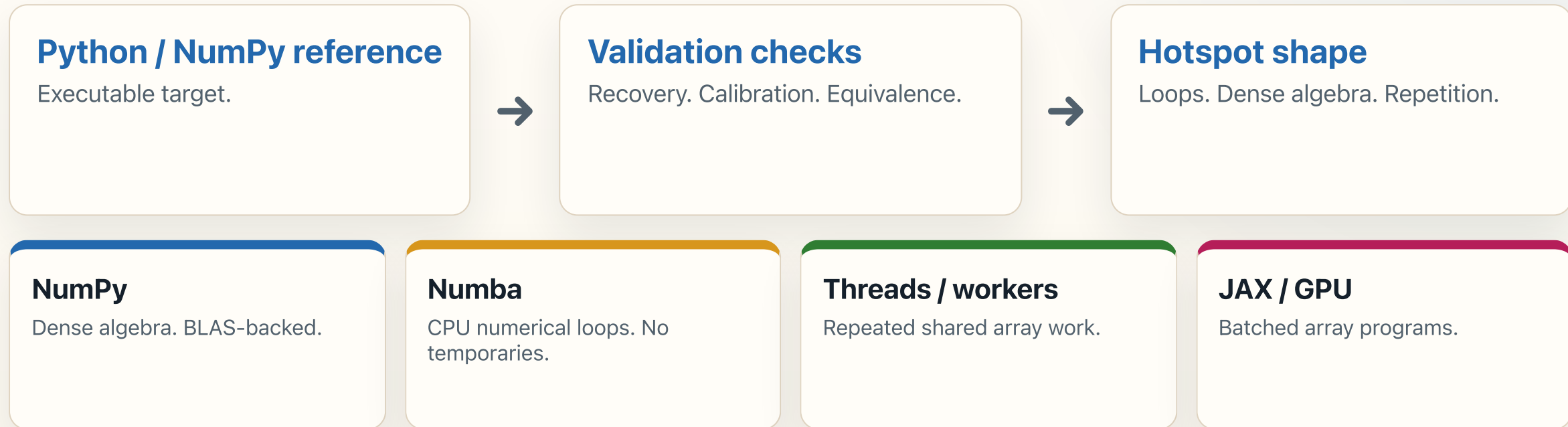
Expected behavior is statistical.

How simulation maps to software testing

Unit tests	Fixed-seed equivalence.
Property tests	Null invariants. Calibration.
Stress tests	Outliers. High dimension.
Load tests	Scale samples. Scale features.
Regression tests	Acceptance criteria pass.

Python stays the control plane.

Keep it readable. Move only hotspots.



Two workloads, two bottleneck shapes

k-means. Iterative fitting.

Assign. Update. Repeat.

Sequential state dependence.

distance loops

temporaries

compiled kernels

Permutations. Resampling inference.

Shuffle. Compute. Repeat.

Independent. High-dimensional limits.

shared arrays

batching

W @ X

Validation. Scale. Acceleration.

MacBook Air

Trust the statistic?



Linux server CPU

How does it scale?



A100 GPU

Does it fit accelerators?

No hidden wins.

Explicit timeouts and OOMs.

Not a leaderboard.

Environments answer different questions.

WORKLOAD 1

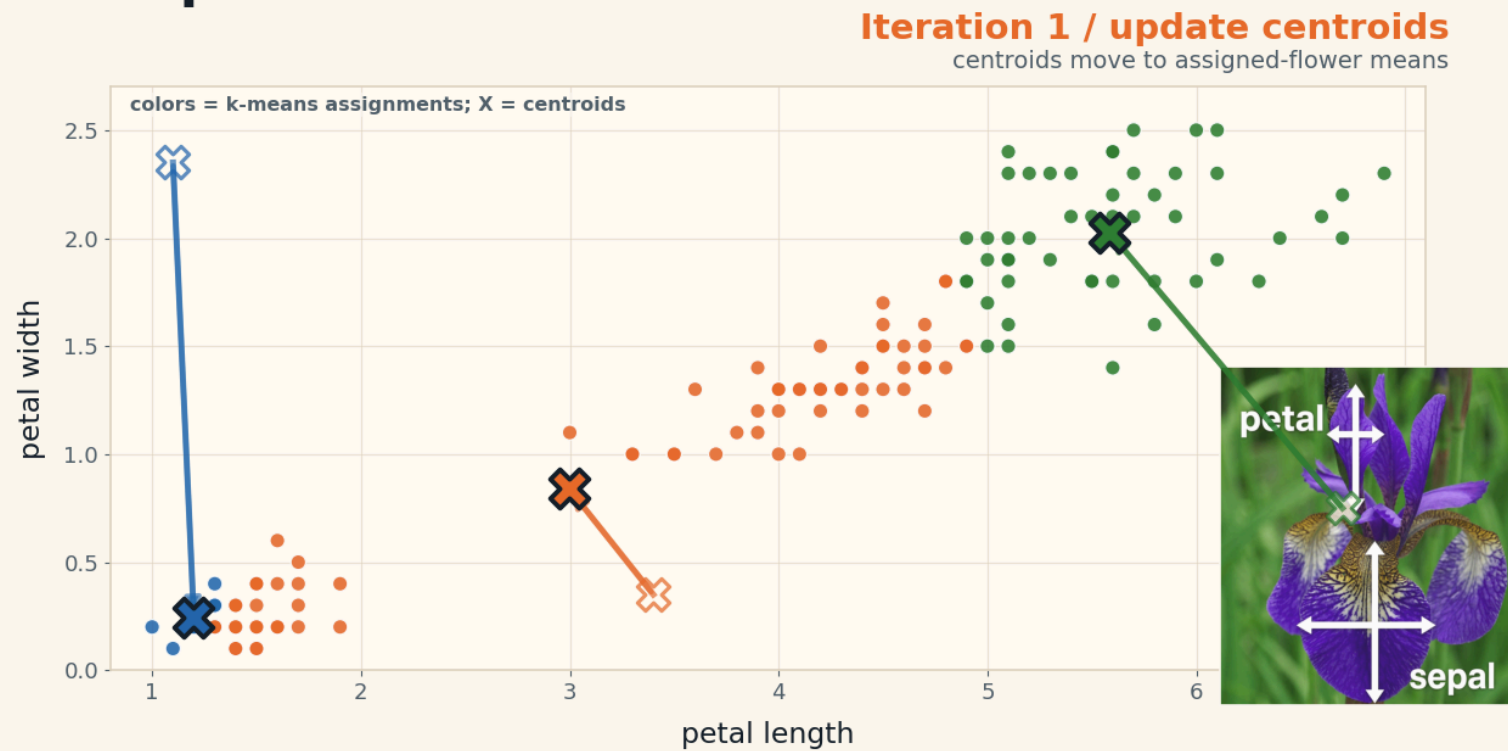
k-means iterative pressure.

Same validation. Vary scale.

K-means movement.

Form clusters. Find centers.

Iris petal measurements



Initialize. Then update.

1. Fixed Start.

Three initial centroids.

2. One Lloyd run

Assign

Nearest centroid.



Update

Assigned mean.

Repeat until stable.

3. Stop.

Assignments stop changing.

Updates depend on previous state.

Labels are just context.

K-means is a useful test.

Exposes broad iterative pressures.

Assignment-update shape.

assign

Nearest centroid.



update

Recompute centroids.



repeat

Stop when stable.

Repeated distance work.

$N \times K \times d$

samples $\times$ clusters $\times$ dimensions.



Sequential state visibility.

Validation contract.

Same data.
Same initialization.
Same stop.
Same objective.

Harder scenarios.

Overlap.
Imbalance.
Outliers.
High dimension.

Bottleneck shapes.

Loops.
Algebra.
Arrays.

Why it generalizes.

Broad iterative pressure.

Pressure: $N \times K \times d$ per iteration.

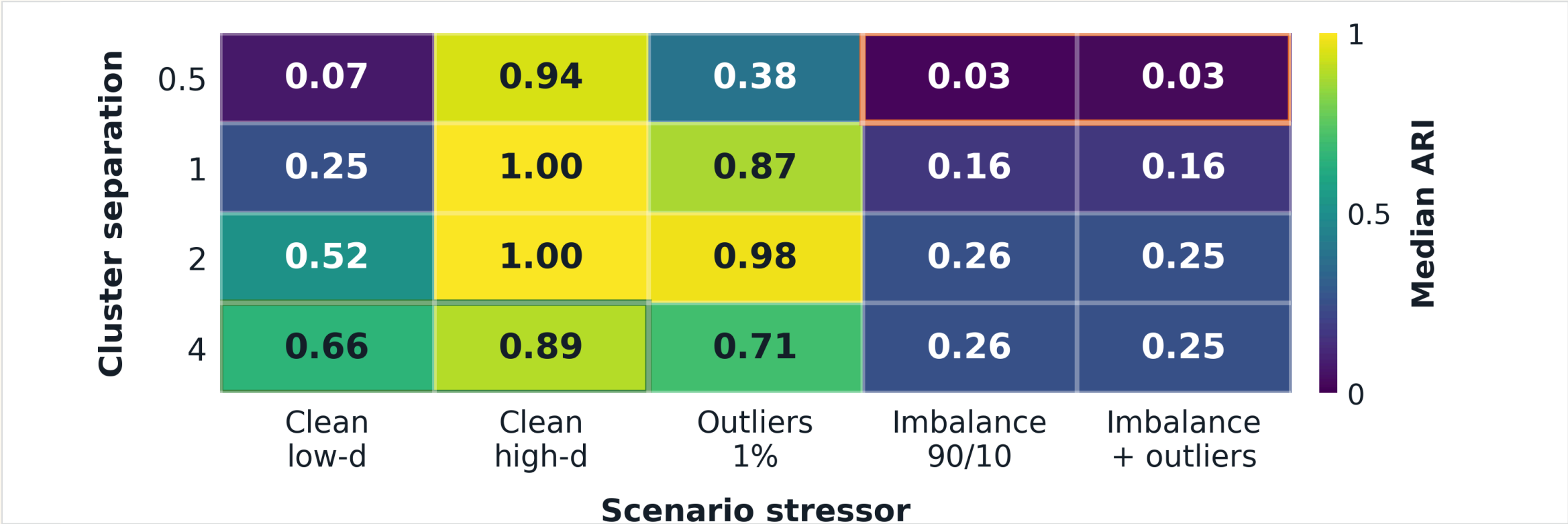
Did k-means recover groups?

Simulation provides the truth.

ARI recovery score.
1 = perfect. 0 $\approx$ random baseline.
Can be negative for worse-than-chance agreement.

Scenario stressors.
Separation. Dimension. Outliers. Imbalance.

Before timing.
Speed fails without recovery.



Scenario difficulty summary.

REFERENCE GATE

Compare against reference.

Time only after matching.

Same data.

Same
initialization.

Same stop.



Readable reference.

Executable oracle.



Numba. NumPy. JAX.

Must preserve results.



Compare inertia + labels.

Stay below tolerance.

MAIN CHECK

3.1e-14

Max inertia diff.

Label agreement: ARI vs reference = 1.0.

TOLERANCE

1e-8

Skip memory-risk rows explicitly.

Three ways to compute.

Same validation. Three expressions.

Numba

Keep explicit loops.

```
@njit(parallel=True)
for i in prange(n):
    best = inf

    for k in range(K):
        dist = sqdist(X[i], C[k])
        if dist < best:
            best = dist
            labels[i] = k
```

Shape: CPU loop-heavy.

NumPy

Dense matrix algebra.

```
x2 = sumsq(X)[: , None]
c2 = sumsq(C)[None, :]

dist2 = x2 + c2 - 2 * (X @ C.T)

labels = argmin(dist2, axis=1)
```

Shape: BLAS-backed algebra.

JAX / GPU

Array expression on device.

```
@jax.jit
def assign(X, C):
    x2 = sumsq(X)[: , None]
    c2 = sumsq(C)[None, :]

    dist2 = x2 + c2 - 2 * (X @ C.T)
    return jnp.argmin(dist2,
axis=1)
```

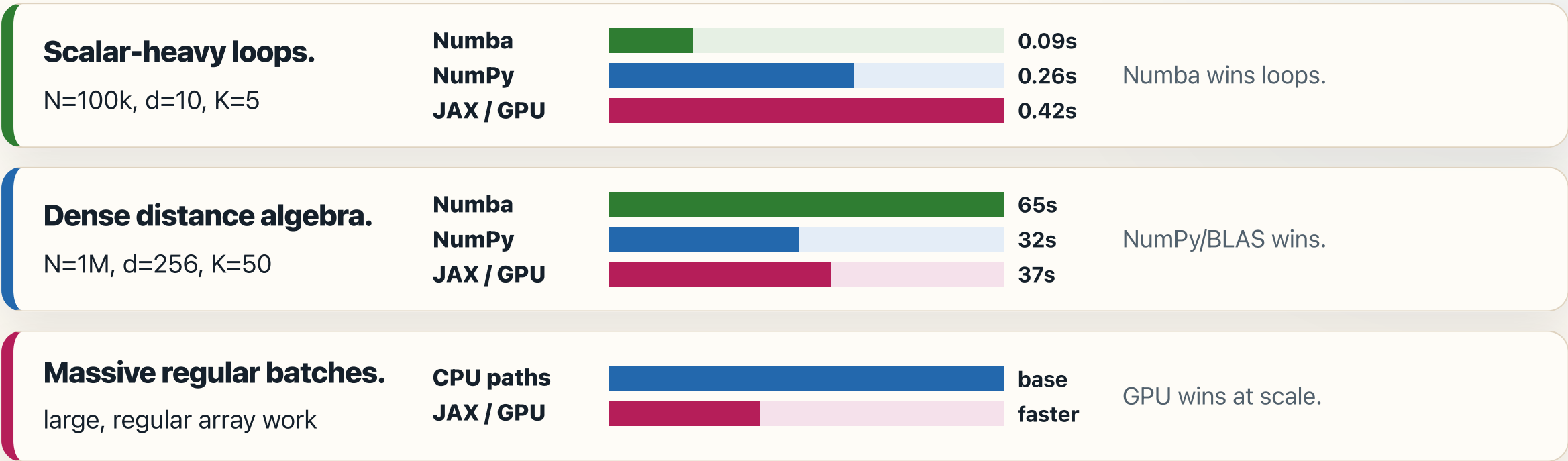
Shape: Batch for accelerators.

Match expression to bottleneck shape.

SCALE TIER

Engine depends on shape.

Read shapes, not rankings.



Shape determines the tool.

K-MEANS TAKEAWAY

Tool follows simulation signal.

Poor recovery

Don't optimize. Debug method.

Scalar loops

Compile explicit CPU loops.

Dense distance algebra

BLAS-backed matrix multiply.

Large regular work

Batch for accelerators.

Applies to broad iterative methods.

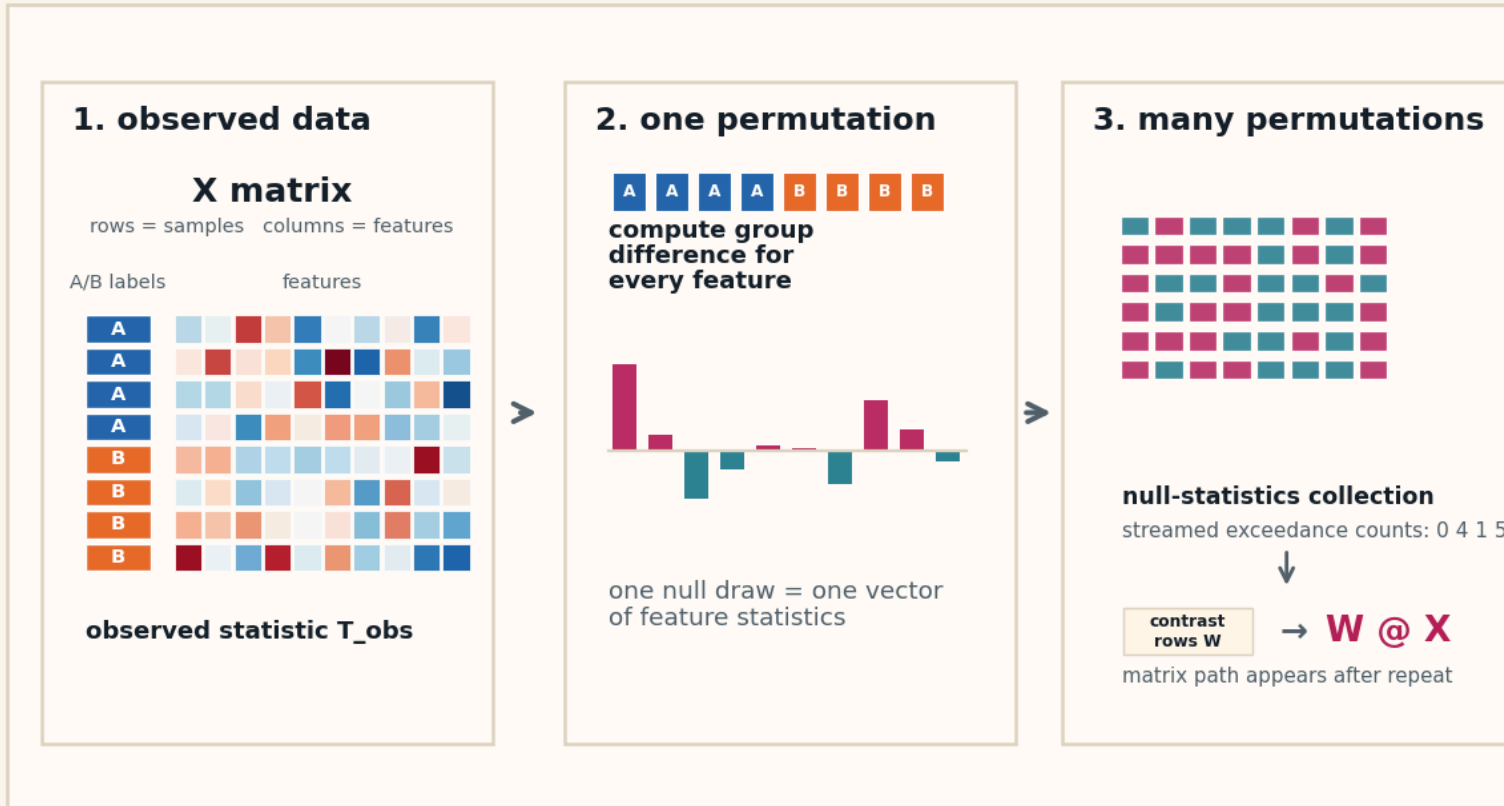
WORKLOAD 2

Permutations. Resampling inference pressure.

Expensive with many features.

Permutation test scaling.

Shuffle labels. Build null.



One resampling loop.

1. Shuffle labels.
2. Compute group differences.
3. Repeat for null.

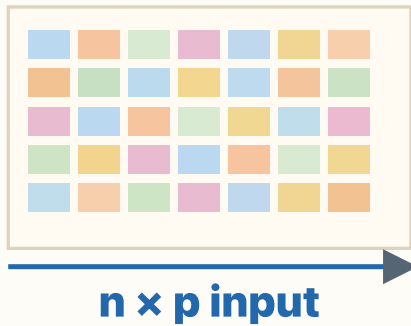
Independent repeats create work.

Why it gets expensive.

Work scales as $R \times p$.

X matrix

n samples.
p features.



samples = patients
features = genes / voxels



R shuffled vectors.

R = null draws.

A B A B A
B A A B B
A A B B A
B A B A A

$\times R$



$R \times p$ statistics.

Or p streamed counts.



Simple math.
Massive workload.

Same math. New shape.

Rewrite the repetition.

Reference loop.

```
for r in range(R):  
    labels_r = same_permutation_stream[r]  
    stat[r, :] = group_difference(X, labels_r)
```

Optimized path.

```
for W_batch in contrast_batches(  
    same_permutation_stream):  
    T = W_batch @ X  
    counts += abs(T) >= abs(T_obs)
```

W_batch: batch_R × n

X: n × p

T: batch_R × p

Stream avoids $R \times p$ matrix.

Equivalence: loop vs. matrix.

Verify before timing.

Same inputs.

Same data.
Same stream.
Same definition.

Reference.

Loop formulation.
One by one.

Optimized path.

Matrix formulation.
Batched computation.

Same result. Faster code.

VALIDATION RESULT

PASS

VALIDATION GRID

18 × 5

workloads × seeds

0.0

max |p diff|

9.4e-16

max |stat diff|

1e-6

validation
tolerance

Acceptance criteria

- ✓ P-values agree exactly.
- ✓ Stat differences are roundoff dust.
- ✓ Optimized path approved.

Null calibration check.

Type-I error near 0.05.

Why calibrate?

Checks inference behavior.

Observed check.

Error estimate ≈ 0.051 .

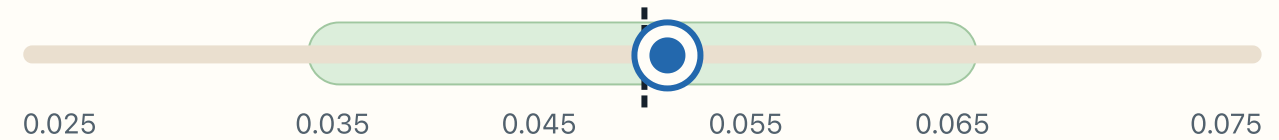
Before scaling.

Acceptable for scaling.

calibrated **0.051**
Type-I error estimate.

Calibration ruler

feature-level binomial band 0.036-0.064



Dashed: Nominal alpha 0.050.

Dot: Observed mean 0.051.

Band is per-feature binomial variation, not CI for the mean.

Calibration checks inference.

Equivalence: code. Calibration: inference.

LOCAL SCALE SIGNAL

When does runtime bend?

p and R reveal bottlenecks.

Same statistic.

Same stream and definition.

EQUIVALENCE PASSED

Local runtime.

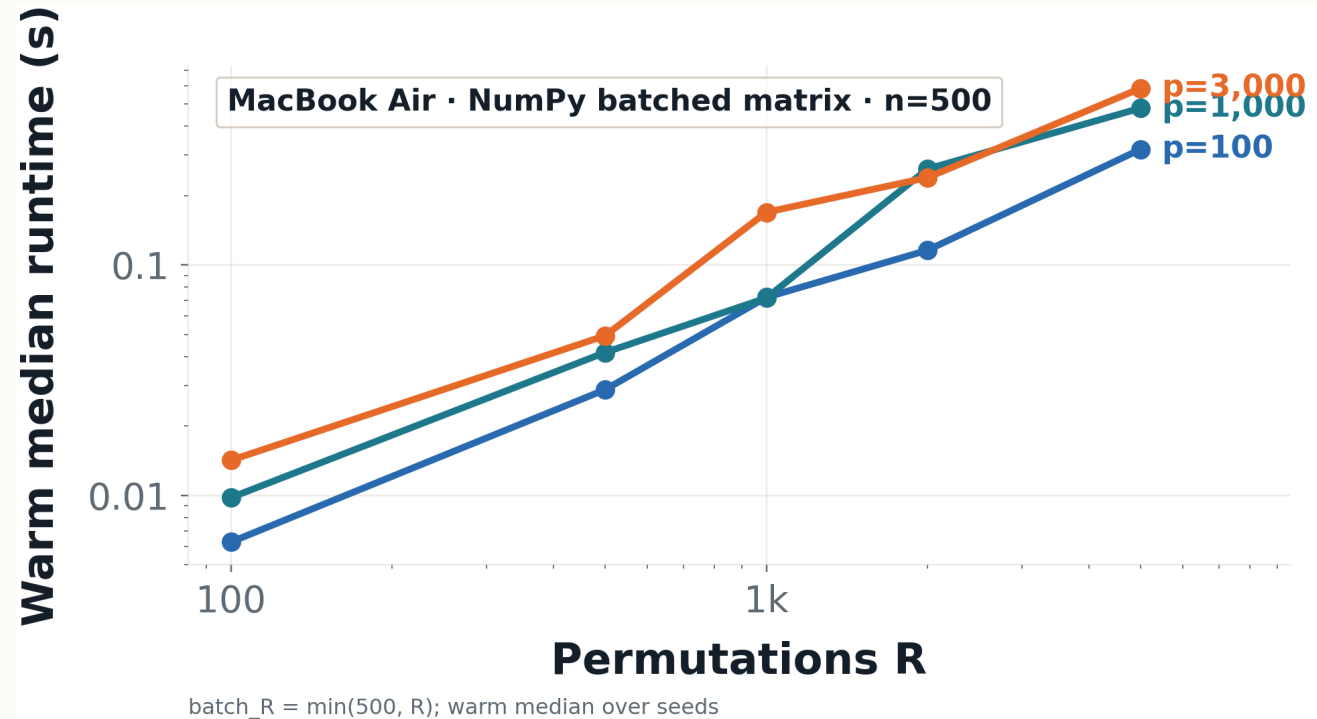
MacBook. Batched NumPy. n=500.

WARM MEDIAN

Scale question.

When does runtime explode?

LOCAL RUNTIME SCALING



MacBook Air local tier · NumPy batched matrix · n=500 · warm median. Shape signal. Not leaderboard.

Local runtime motivates server/GPU.

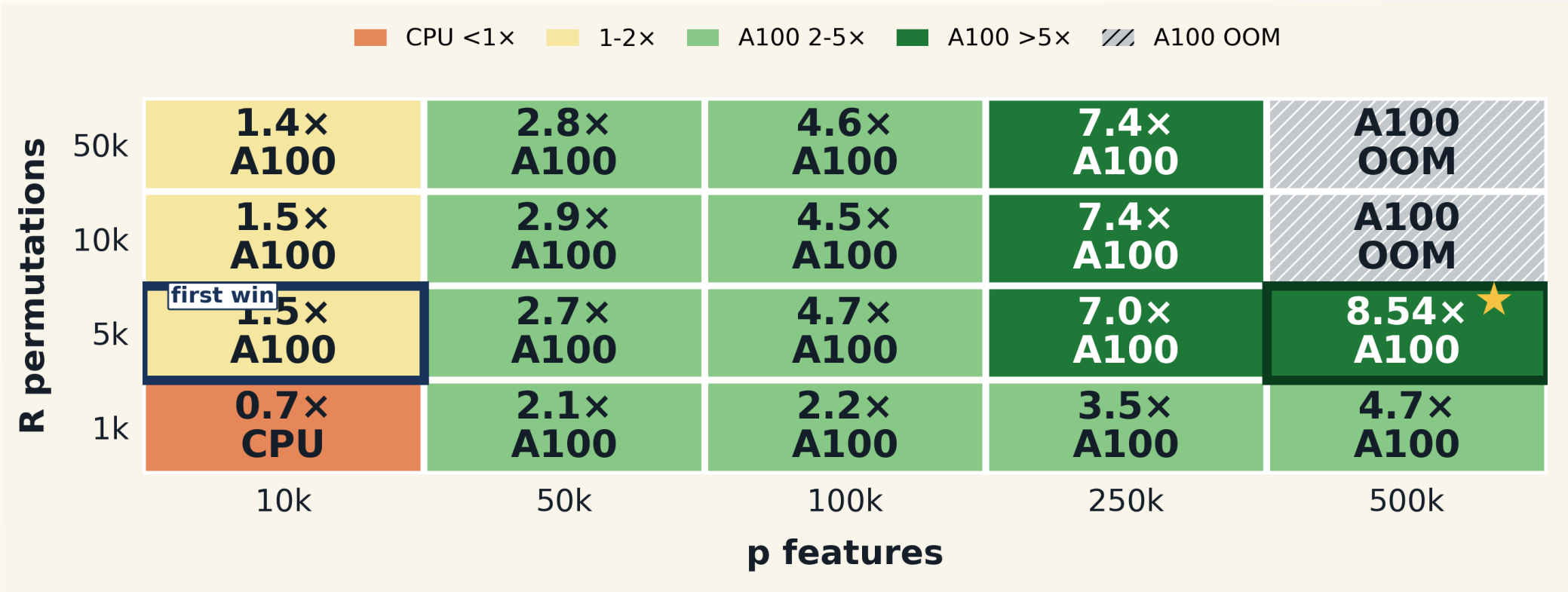
When do GPUs help?

Requires massive R × p.

Scope
Speedup: CPU vs. GPU e2e.

On this A100 pipeline.
p=10k. R=5k. batch=8192.

Explicit OOM cells.
Unavailable. Not CPU wins.



Transfer included; compile excluded. Not a leaderboard.

Validation and batching unlock GPUs.

PARALLELISM

Parallelism helps. Then flattens.

Measure time and marginal gain.

Total time drops.

Parallelism helped early.

runtime

Returns flatten.

Gains shrink later.

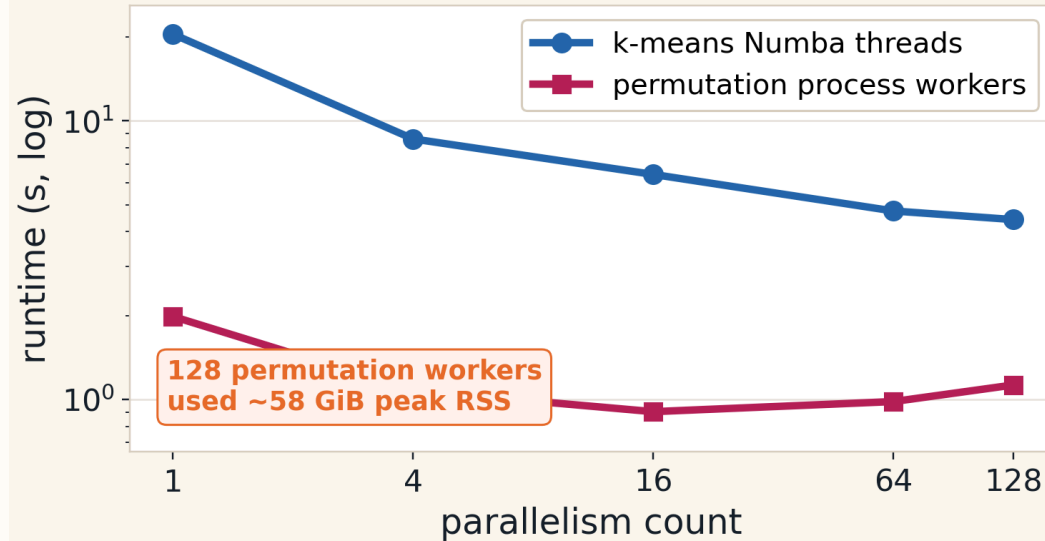
marginal gain

Memory matters.

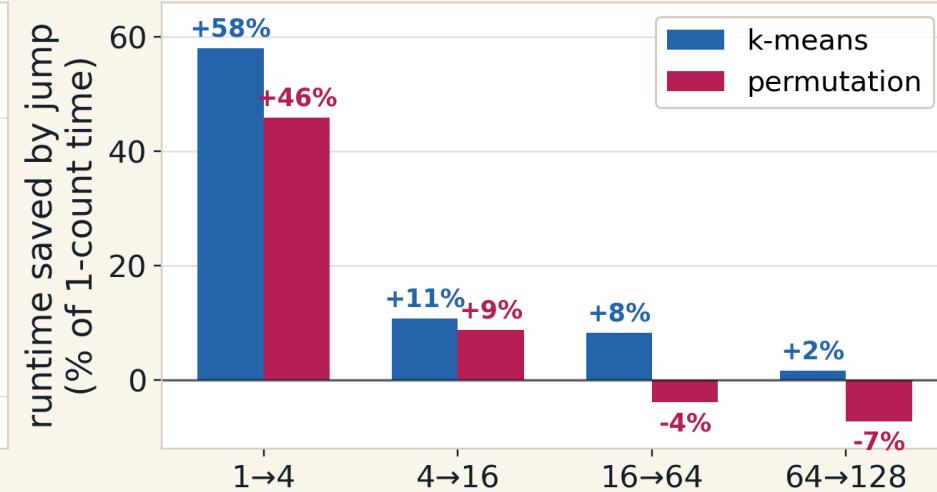
Workers consume memory.

memory

A. Total time



B. Diminishing returns



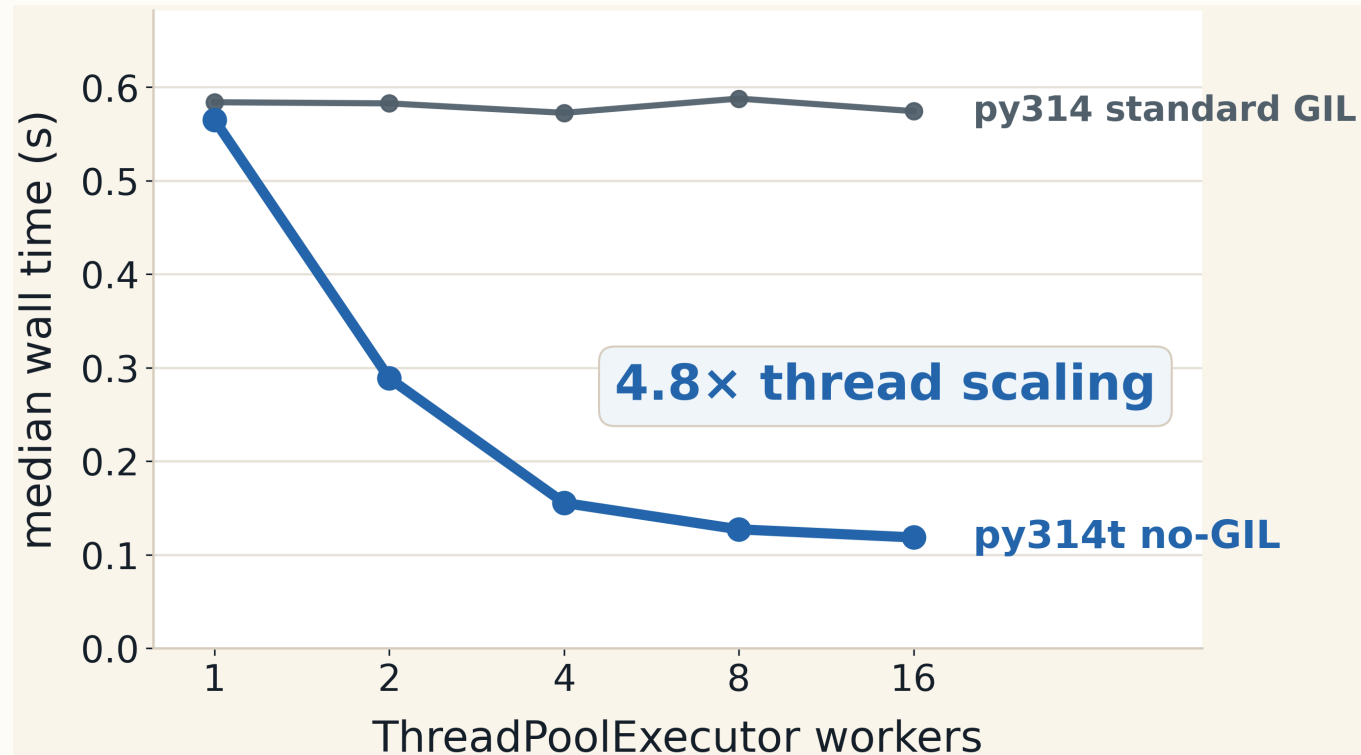
Execution model matters next.

Capture early speedup.

EXECUTION MODEL

Free-threaded CPython changes this workload's tradeoff.

Threads become viable when the workload is designed for shared-memory parallelism.



Standard GIL builds

~1.02x

1 → 16 workers

py314 stays flat

CPython 3.14t / free-threaded

4.77x

1 → 16 workers

threads scale this workload

Execution-model payoff

2.43x faster

32% less peak RSS

ThreadPool vs ProcessPool

Official term: free-threaded CPython build, available since Python 3.13; supported but optional, not default.

It can run Python threads on CPU cores in parallel; C extensions may re-enable the GIL, and single-thread overhead exists.

CONNECTING THE TOOLS

What each tool solved.

Match tool to shape.

Numba

Bottleneck:
CPU loops.

Solved: K-means assignment.

NumPy / BLAS

Bottleneck:
dense algebra.

Solved: Distance identity.

Threads / workers

Bottleneck:
shared data repeats.

Solved: Permutation workers.

JAX / GPU

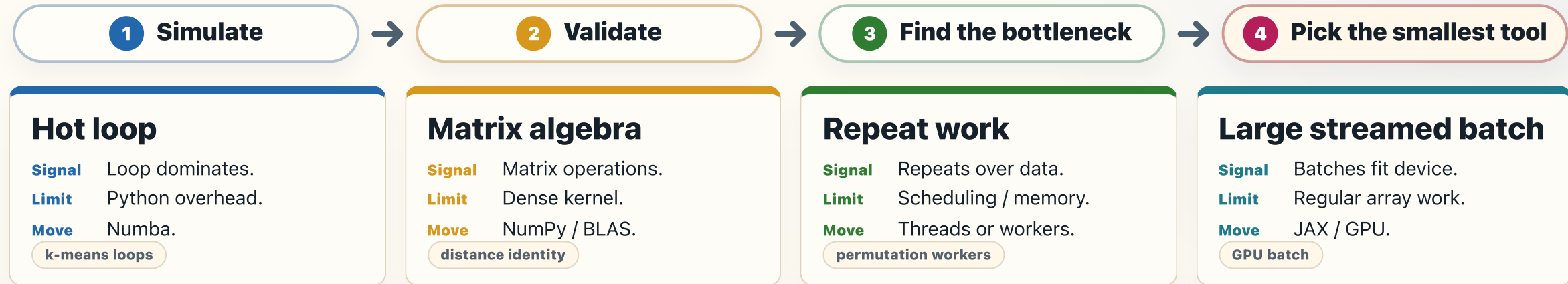
Bottleneck:
device-resident batches.

Solved: Streamed $W @ X$.

Same statistic. Different tool.

Evidence to tool.

Follow bottlenecks, not logos.

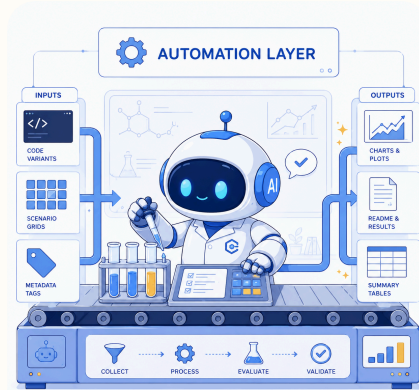


Validate first. Move bottlenecks.

AI scales the workflow. We own the claims.

AI automation.

- Implementation variants.
- Scenario runners.
- Metadata capture.
- Plot regeneration.
- Result manifests.



Scientific judgment.

- Statistical target.
- Data assumptions.
- Validation criteria.
- Hard-case interpretation.
- Scientific claims.



AI automates. Humans claim.

CLOSE

**Make the statistic testable.
Then make the bottleneck fast.**

Clear enough to validate. Fast enough to scale.

THANK YOU

Questions?

Slides. Reproducible artifacts.

`github.com/LucaJiang/FastStatisticalModels4Python`



Slides. Reproducible artifacts.

BACKUP: EVIDENCE SCOPE.

Evidence map. Different questions.

Why three environments?

MacBook = validation.

Equivalence. Calibration. Local shape.

Server = scale. Parallelism.

Shape scaling. Memory sweeps.

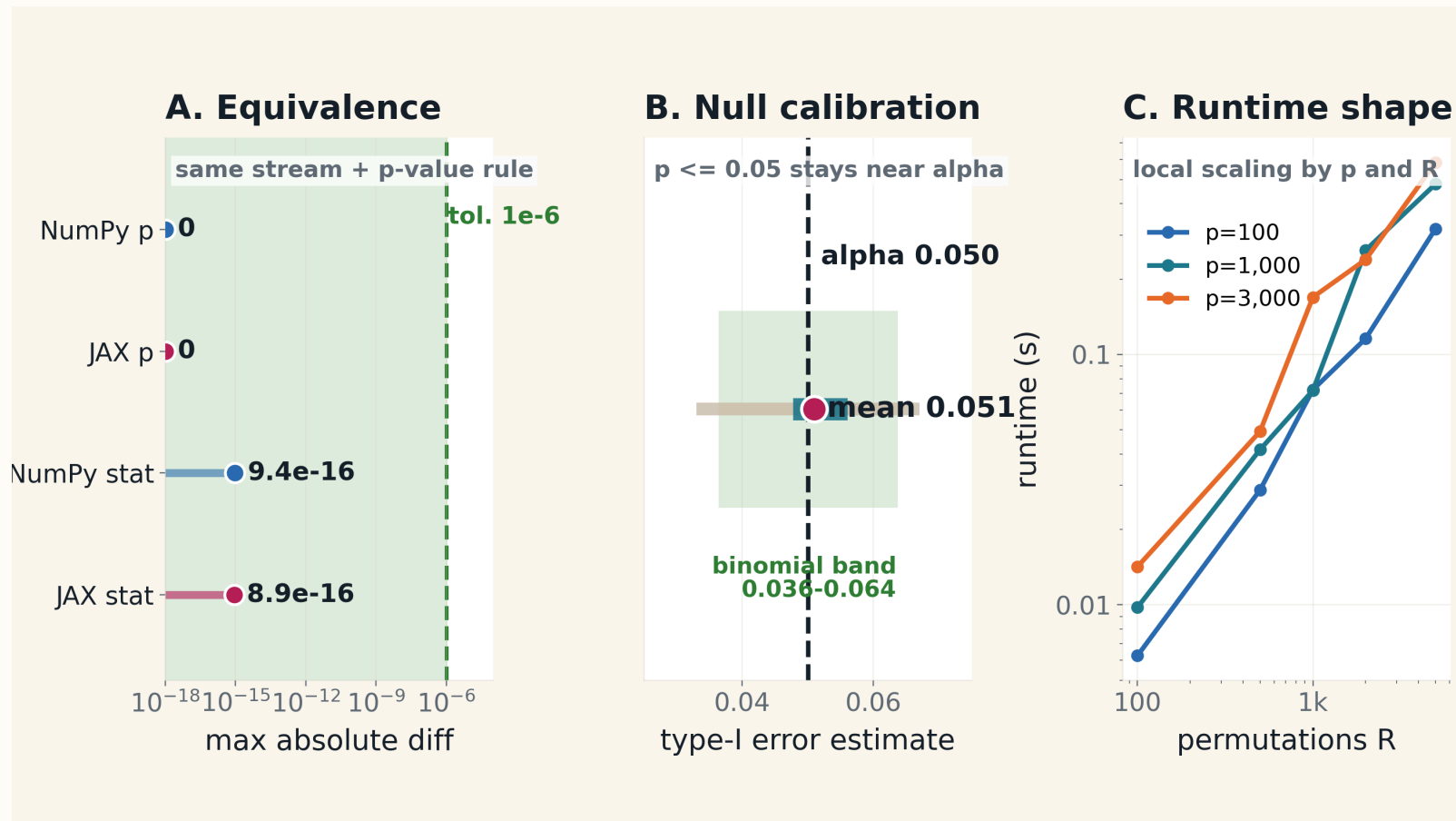
GPU = accelerator pipeline.

Break-even evidence. Streamed reduction.

Detailed metrics in repository.

Validation inventory.

Code. Inference. Runtime shape.



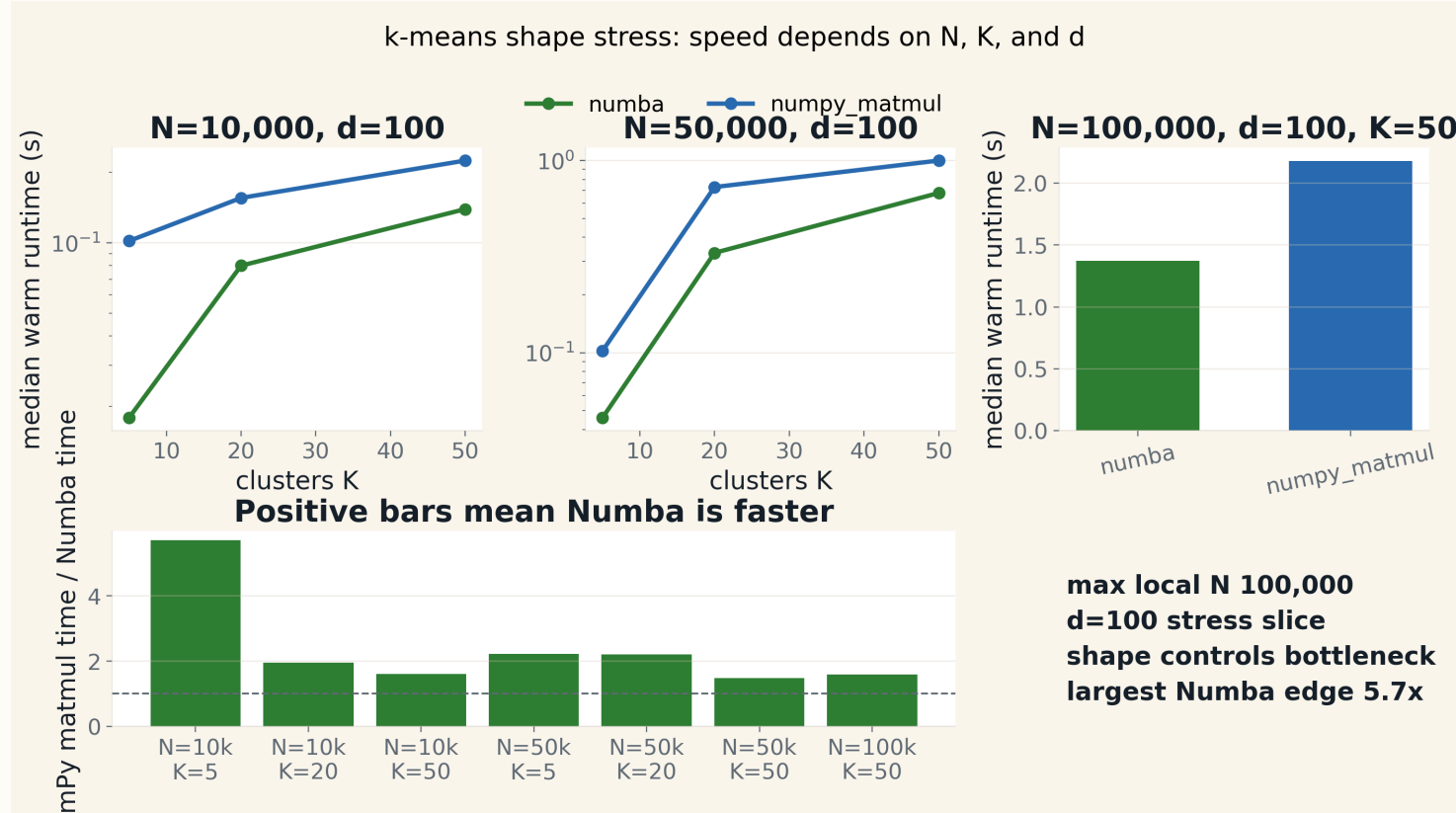
Numba wins loop shapes.

Q&A scope

Question: When does Numba win?

Local tier: MacBook runs.

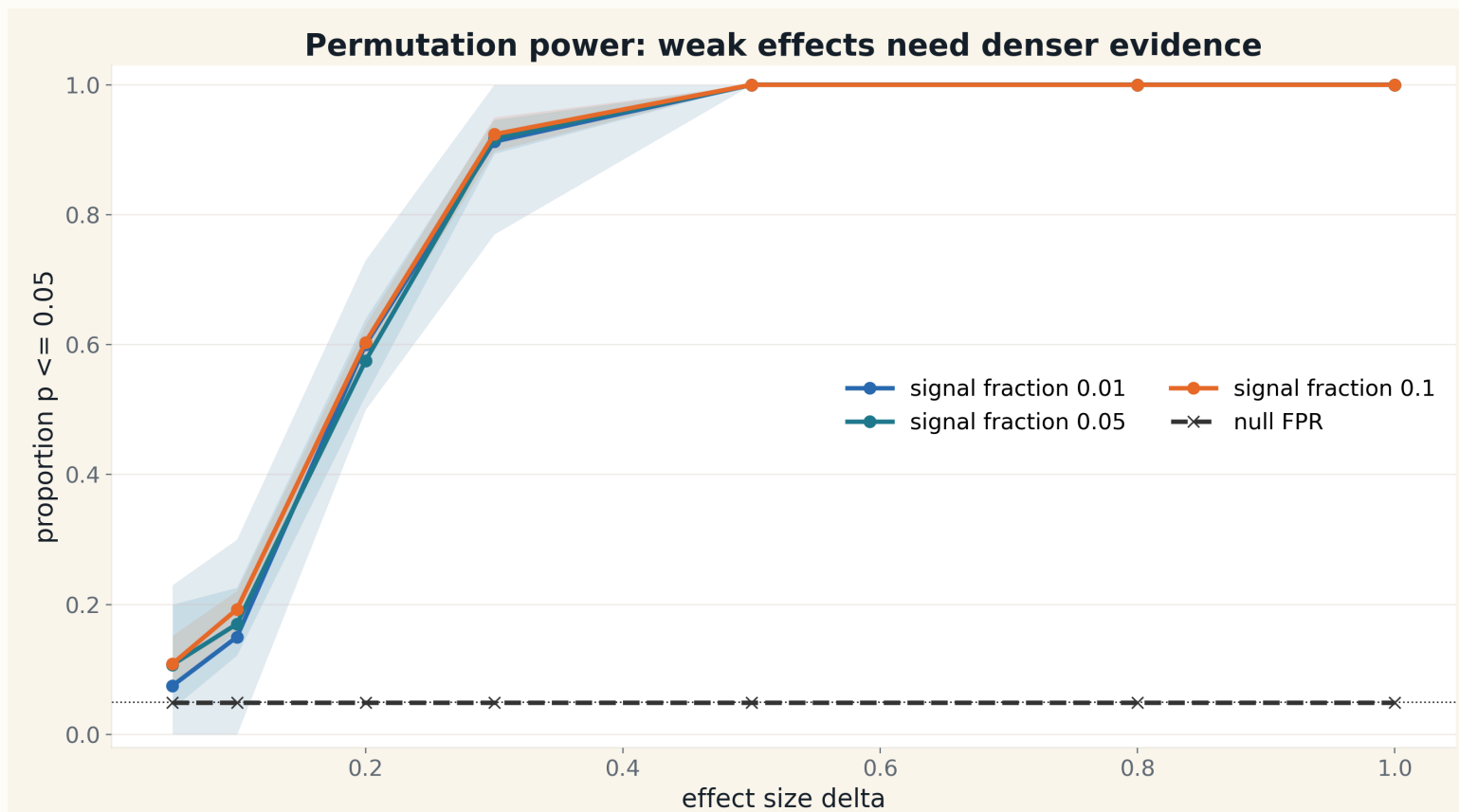
Answer: When explicit loops dominate.



Loops favor Numba. Algebra favors BLAS.

Power vs. effect size.

Do we detect strong signals?



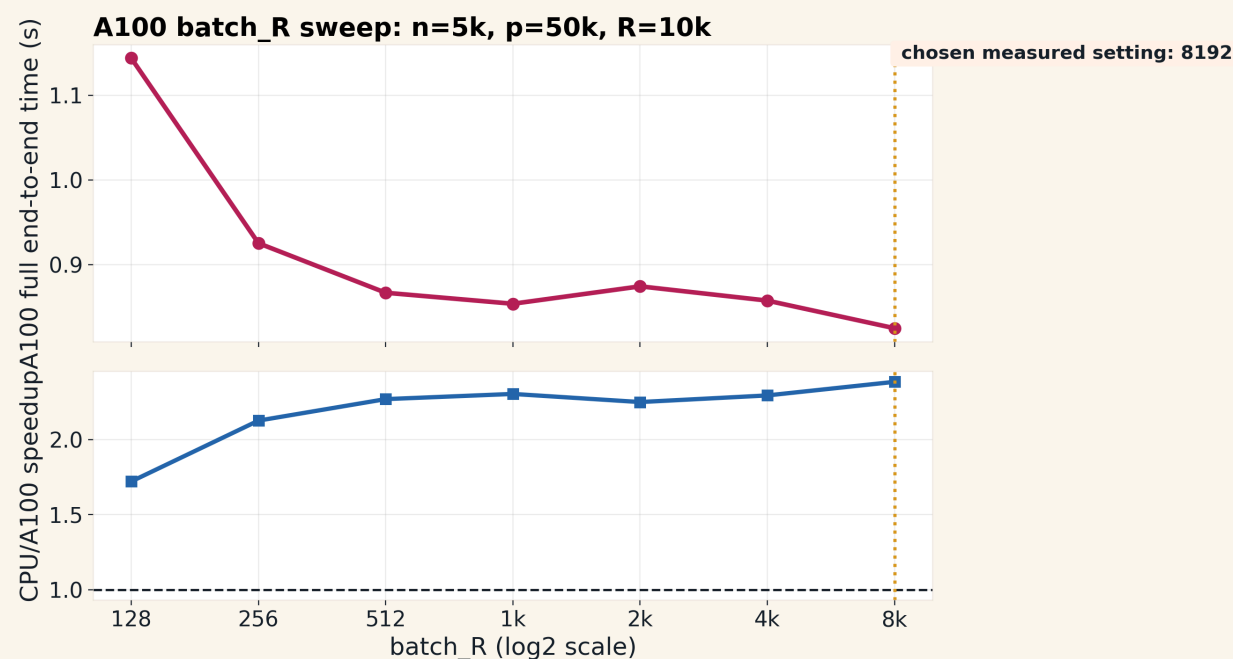
Strong signals yield high power.

BACKUP: BATCH_R=8192.

BACKUP: GPU TUNING

Backup: what batch_R tuning showed

Larger batches amortized overhead; 8192 was fastest only within this measured grid.



Definition

batch_R = permutation rows processed in one A100 batch.

Trend

Small batches were slower; larger batches reduced stream overhead.

Diminishing returns

Most of the gain appeared by 512-1024; later points were nearly flat.

Claim boundary

8192 was fastest in this measured grid. Larger values were not tested here.

Timing improved most at small batch_R; after 512-1024 the curve mostly flattened. 8192 is the fastest accepted value measured here.

batch_R tunes pipeline overhead; it does not change the statistical question.

BACKUP: AI WORKFLOW.

What the agent automated.

Experiment orchestration.

Runners. Scenarios. Manifests.

Visualization surface.

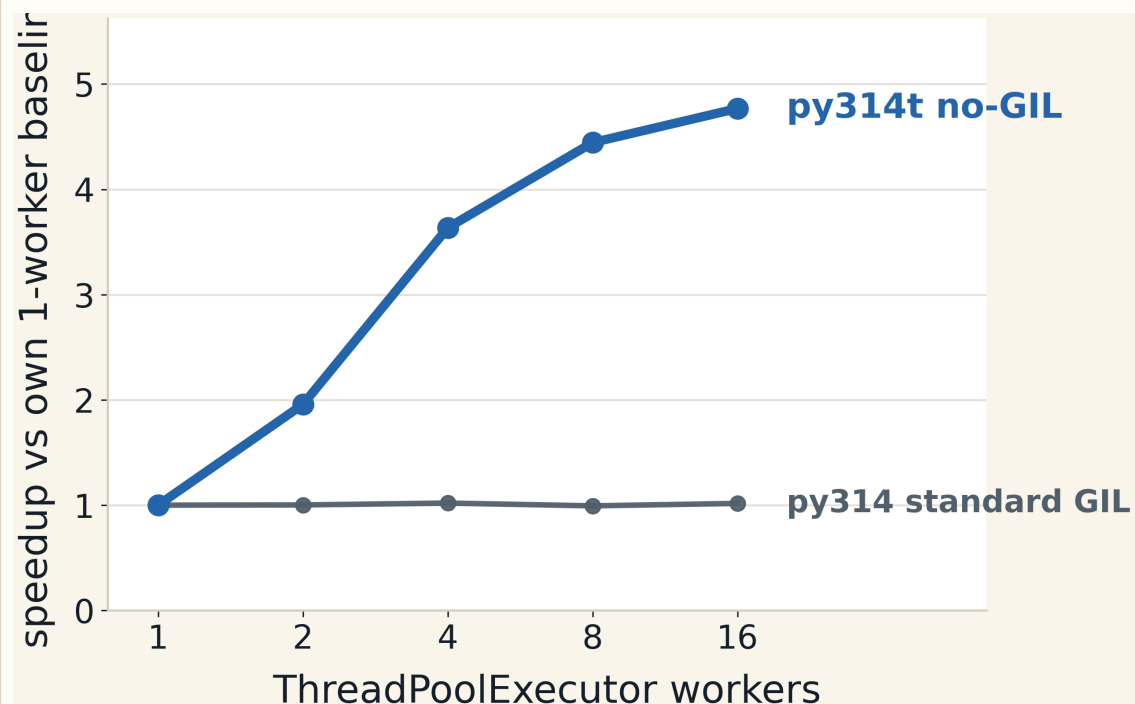
Regenerated figures from CSVs.

Repository surface.

Commands. Summaries. QA notes.

AI automated. Humans claimed.

Scaling evidence.



Compare scaling, not base speed.

ThreadPoolExecutor workers

Build	1 worker	16 workers	Ratio
py314 standard GIL	0.584s	0.574s	1.02x
CPython 3.14t free-threaded	0.565s	0.119s	4.77x

CPython 3.14 answers the standard-GIL baseline; CPython 3.14t answers the free-threaded execution-model question.

ThreadPool vs ProcessPool runtime

py314 ProcessPool	0.571s
3.14t ThreadPool	0.235s
2.43x faster	

ThreadPool vs ProcessPool peak RSS

py314 ProcessPool	1.017 GiB
3.14t ThreadPool	0.692 GiB
32% lower	

ThreadPool shares dataset.